```
class Camera extends Product
  render: ->
    @view.innerHTML = """
    #{@name}: #{@info.stock}
    {@gallery.render()}
    """
```

A heredoc to output the name and the gallery. The gallery output is generated by the render method of the gallery object.

The `constructor` and the `render` method have both been overridden:

```
class Camera extends Product
  constructor: (name, info) ->
    @name = name
    @info = info
    @gallery = new Gallery
  render: ->
    @view.innerHTML = """
    #{@name}: #{@info.stock}
    {@gallery.render()}
  """
```

Create a gallery from the Gallery class.

Get the gallery to render.

To override a method that you got from extending a class, declare a new version in the new class. Sounds too easy, and it is. What happens when you create some cameras and try to find them?

```
class Product

Product.find = (query) ->
  (product for product in products when product.name is query)

class Camera extends Product

x1 = new Camera 'X1', {}
Product.find 'X1'
# []
```

The camera was not found! The list of product instances is updated inside the `Product` constructor. Once you define a new constructor for the `Camera` class, the `Product` constructor won't be invoked when you `new` a `Camera`. No `Camera` instances will be added to the array. There's a solution to this problem: the `super` keyword.

5.4.2. super